

FPGA 설계

Term project

<Lap 기능을 추가한 Stopwatch>

학번: 20211494, 20232928

이름: 윤중현, 오민재

목차

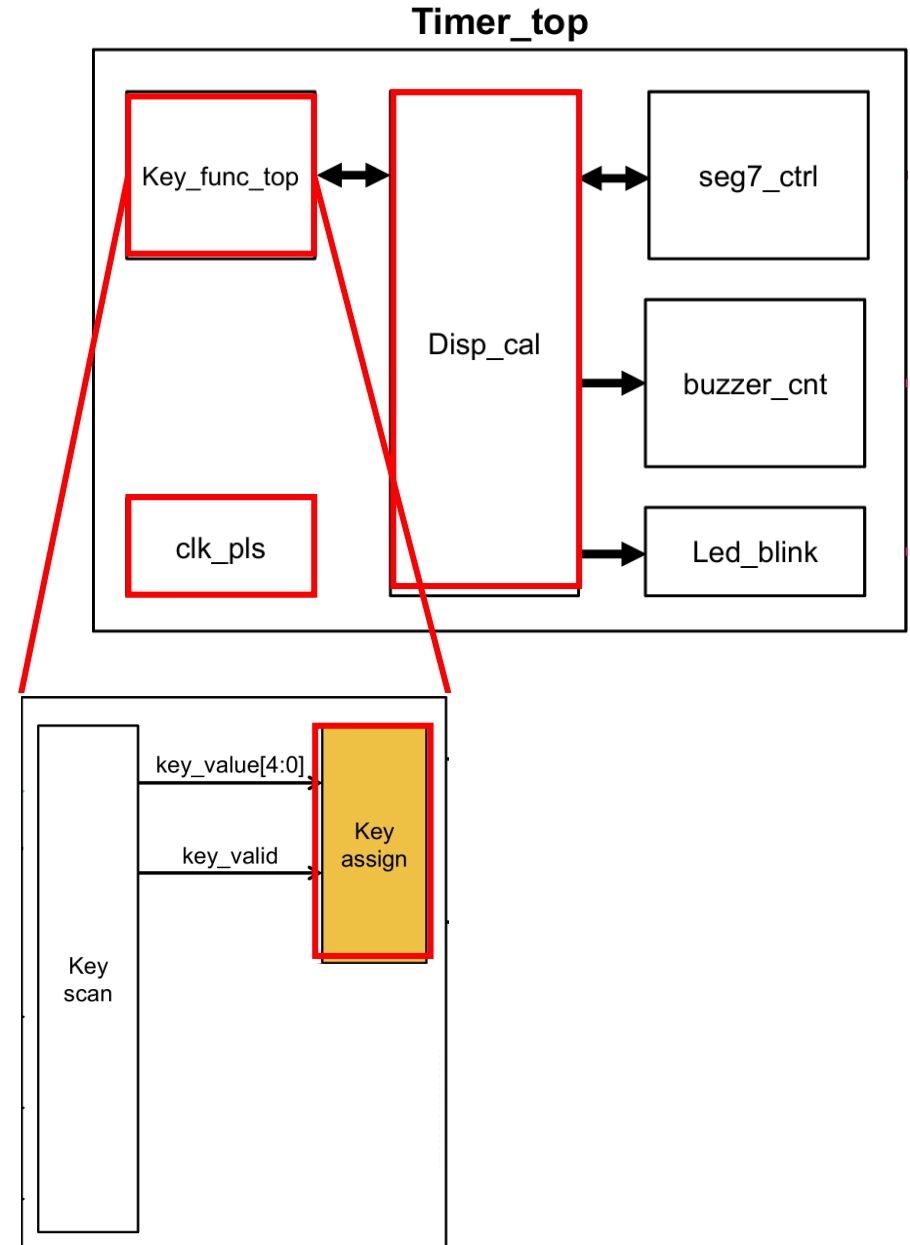
1. 시나리오
2. 역할 분담
3. 코드 설명
4. 시뮬레이션 검증
5. FPGA 보드 검증

1. 시나리오

기존 타이머에 stopwatch 기능과 더불어 stopwatch의 표기 시간을 저장하는 기능(랩)을 추가 하여 timer와 stopwatch 을 탑재한 다중기능의 보드 설계

구현 기능)

- Timer ↔ stopwatch
: 양방향 전환 가능한 버튼 생성
- Lap (저장기능)
: stopwatch의 시간을 레지스터에 저장
- Lap 개수 led 표시
: 저장된 랩 타임 숫자만큼 led 불빛 개수로 표시
- Lap time display
: 앞서 저장된 디스플레이로 송출

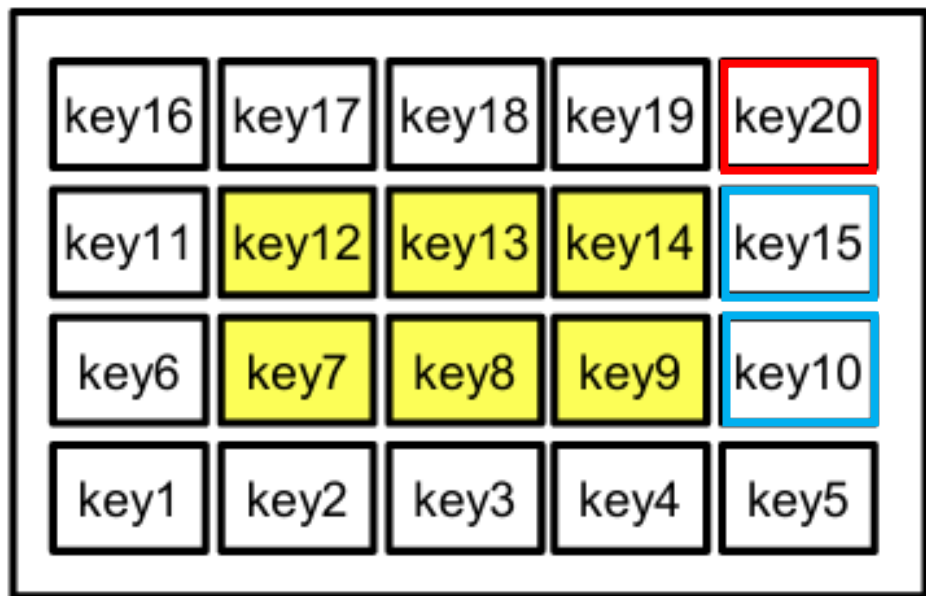


1. 시나리오

Timer 기능에서 활용하지 않은 Key button으로 구현하고자 하는 기능 추가하기

구현 기능)

- **Key 20** (Timer, stopwatch)
: timer와 stopwatch의 모드 전환
- **Key 15** (Stopwatch)
: stopwatch의 현재 시간을 저장(Lap)
- **Key 10** (Stopwatch)
: 저장된 랩 타임을 7-segment에 표시



2. 역할 분담

윤중현:

1. Stopwatch 기능 추가
2. Lap기능 추가

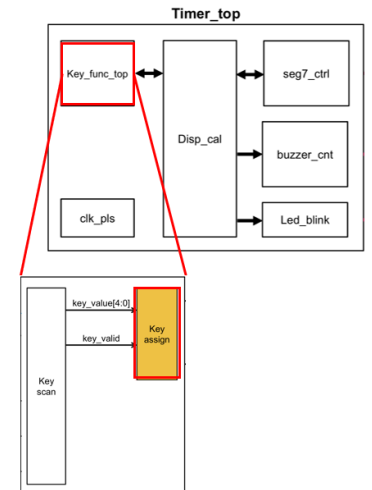
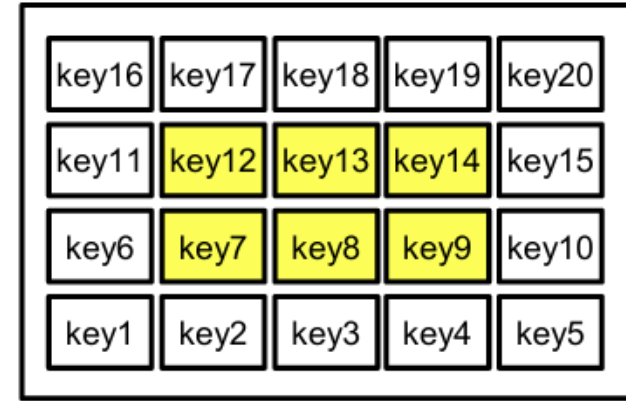
오민재:

3. Lap time 저장 기능 추가
4. Lap time 7-segment에 표기

3. Code 설명

1. key_assign.v: Key20(F1) & Key15(F2) 활성화

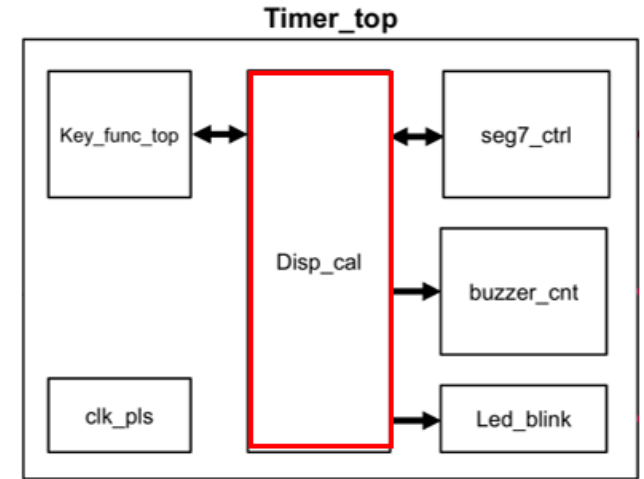
```
3 module key_assign(  
4     input        i_rstn    ,  
5     input        i_clk     ,  
6     input        i_key_valid,  
7     input  [4:0] i_key_value,  
8     output  [4:0] o_bcd_data ,  
9     output        o_key_valid,  
10  
11     // [추가] 출력 포트: mode와 lap 신호를 내보내기  
12     output reg   o_mode_key ,  
13     output reg   o_lap_key  ,  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28     else if(i_key_value==5'd14) r_bcd_data <= 5'h3;  
29     else if(i_key_value==5'd7)  r_bcd_data <= 5'h4;  
30     else if(i_key_value==5'd8)  r_bcd_data <= 5'h5;  
31     else if(i_key_value==5'd9)  r_bcd_data <= 5'h6;  
32     // [추가] 새로운 키 매핑 (F1, F2 키 사용)  
33     else if(i_key_value==5'd20) begin  
34         o_mode_key <= 1; // F1 누르면 모드 신호 발생  
35         r_bcd_data <= 5'hf; // 숫자가 아니므로 f  
36     end  
37     else if(i_key_value==5'd15) begin  
38         o_lap_key <= 1;  // F2 누르면 랩 신호 발생  
39         r_bcd_data <= 5'hf;  
40     end
```



- 실제 보드 혹은 시뮬레이션에서 Key 20과 Key 15가 눌렸을 때, 추가한 기능을 구현하기 위한 output 신호들을 선언
- Key 20 은 보드를 Timer와 Stopwatch 기능을 서로 바꾸는 역할. (mode 전환)
- Key 15 는 Stopwatch에서 시간을 기록하는 명령어를 수행할 수 있는 신호를 발생. (Lap 기능)

2. Disp_cal.v: Stopwatch와 Lap 구현 알고리즘 추가

```
3 module disp_cal (
4     input      i_rstn    ,
5     input      i_clk     ,
6     input      i_pls_1k  ,
7     input      i_key_valid ,
8     input      i_start   ,
9     input [ 3:0] i_bcd_data ,
10    output [31:0] o_bcd8d  ,
11    output      o_fin     ,
12    //추가한 코드
13    //스탑워치용 입력 포트
14    input i_mode_sw , //0: Timer, 1: Stopwatch
15    input i_lap_sw , //Lap Time 버튼
16    input i_pls_10ms //10ms pulse(clk_pls에서 받아들임)
17
18    else if (r_start_en) begin
19        if(i_mode_sw == 0) begin
20            // Timer 모드 (Count Down)
21            if(r_cnt_sec == 0) begin
22                r_fin <= 1'b1;
23            end
24            else begin
25                if(r_time_sec == 27'd999999) begin
26                    r_time_sec <= 27'd0;
27                    r_cnt_sec <= r_cnt_sec - 1;
28                end
29                else begin
30                    r_time_sec <= r_time_sec + 1;
31                end
32            end
33        end
34        else begin
35            // Stopwatch 모드 (Count Up)
36            if(i_pls_10ms) begin
37                // 최대값 체크 (99분 59초 99)
38                if(r_stop_cnt < 24'd599999) begin
39                    r_stop_cnt <= r_stop_cnt + 1;
40                end
41            end
42        end
43    end
44 end
```



- Mode 를 변경하는 신호를 받으면 stopwatch로 바꾸는 코드
- 앞선 Key_assig에서 i_mode_sw 신호를 수신하고 0이면 Timer이고, 1이면 Stopwatch로 동작.
- i_mode_sw == 1일 때, i_pls_10ms를 입력으로 하여 10ms씩 count up하는 기능 추가.

```

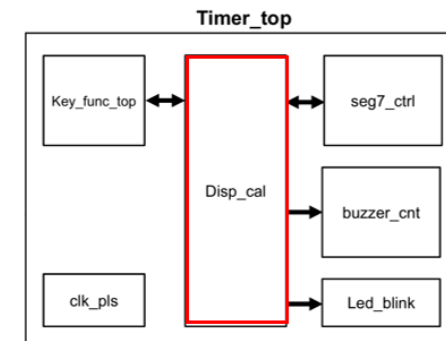
74 // [추가] Lap Time 버튼 처리 (Rising Edge)
75 always @(posedge i_clk or negedge i_rstn) begin
76     if(!i_rstn) begin
77         r_lap_en <= 0;
78         r_lap_sw_d <= 0;
79         r_lap_data <= 0;
80     end
81     else begin
82         r_lap_sw_d <= i_lap_sw;
83         if (i_lap_sw && !r_lap_sw_d) begin // 버튼 누른 순간
84             r_lap_en <= ~r_lap_en; // 모드 토글
85             if (r_lap_en == 0) begin
86                 // 현재 스톱워치 시간을 '찰칵' 저장
87                 r_lap_data <= r_stop_cnt;
88             end
89         end
90     end
91 end

```

```

174 // 최종 출력: 모드와 랩타임 상태에 따라 결정
175 assign o_bcd8d = (i_mode_sw==0) ? w_timer_disp : //timer mode
176                 (r_lap_en)      ? r_lap_data   : // Stopwatch Lap mode
177                 r_stop_cnt      ; // Stopwatch 진행 모드
178 assign o_fin = r_fin & (!r_fin_d);

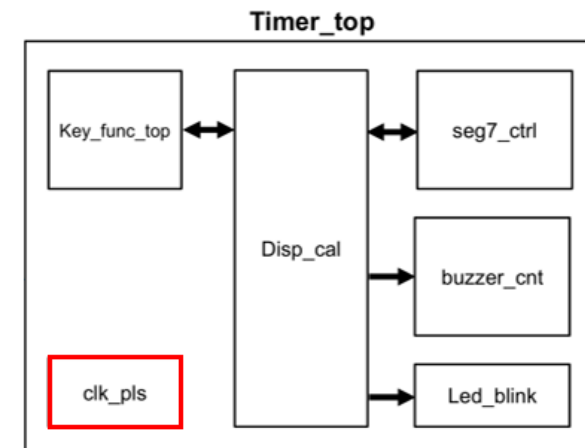
```



- Lap 하는 신호를 받으면 현재 display되고 있는 값을 저장.
- 앞선 Key_assig에서 i_lap_sw 신호를 수신하고 버튼이 눌리는 순간, Toggle하여 count up 되고있는 값을 저장.
- 7-Segment로 보내는 값이 Mode에 따라 timer 혹은 stopwatch 값을 보내고, Stopwatch 내에서 Lap 신호에 따라, count up값 혹은 lap time 값을 출력

3. clk_pls.v: 10ms(0.01초) 단위로 pulse 생성

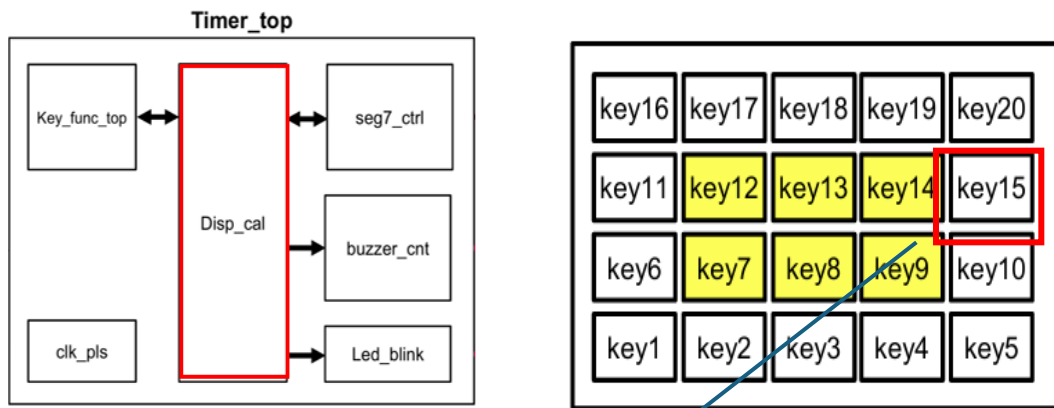
```
3 module clk_pls(  
4     input i_clk,  
5     input i_rstn,  
6     output o_pls_1k,    //1ms pulse  
7     output o_pls_10ms   //10ms pulse(for stopwatch)  
8 );  
37 //10ms 펄스 생성  
38 always@(posedge i_clk, negedge i_rstn) begin  
39     if(!i_rstn) begin  
40         cnt_10ms <= 4'd0;  
41         pls_10ms <= 1'b0;  
42     end  
43     else begin  
44         pls_10ms <= 1'b0;  
45         //1ms 펄스를 10번 count하여 10ms 펄스 생성  
46         if(pls_1k == 1'b1) begin  
47             if(cnt_10ms==4'd9) begin  
48                 cnt_10ms <= 4'd0;  
49                 pls_10ms <= 1'b1;  
50             end  
51             else begin  
52                 cnt_10ms <= cnt_10ms + 1;  
53             end  
54         end  
55     end  
56 end  
57 assign o_pls_1k    = pls_1k;    //1ms clk  
58 assign o_pls_10ms = pls_10ms; //10ms clk
```



- 기존에 있는 1ms 를 생성하는 pulse를 이용하여, 10ms를 생성하는 pulse를 제작.
- **pls_1k**와 4bit의 register인 **cnt_10ms**를 이용하여, pls_10ms를 생성.
- pls_10ms을 Disp_cal에서 count up 코드와 동기 시키면, 0.01초씩 증가하는 stopwatch를 구현.

4. Lap time 저장

```
94 always @(posedge i_clk or negedge i_rstn) begin
95     if(!i_rstn) begin
96         r_lap_sw_d <= 0;
97         r_lap_count <= 0;
98         r_lap_time[0] <= 0;
99         r_lap_time[1] <= 0;
100        r_lap_time[2] <= 0;
101        r_lap_time[3] <= 0;
102        r_lap_time[4] <= 0;
103        r_lap_time[5] <= 0;
104        r_lap_time[6] <= 0;
105        r_lap_time[7] <= 0;
106    end
107    else begin
108        r_lap_sw_d <= i_lap_sw;
109        // LAP 버튼 rising edge 검출
110        if (i_lap_sw && !r_lap_sw_d && i_mode_sw) begin
111            // 8개 저장
112            if (r_lap_count <= 8) begin
113                r_lap_time[r_lap_count] <= r_stop_cnt;
114                r_lap_count <= r_lap_count + 1;
115            end
116        end
117        // 스탑워치 정지 시 (start_en이 0이 되면) 랩 초기화
118        if (!r_start_en && i_mode_sw) begin
119            r_lap_count <= 0;
120        end
121    end
122 end
```



- LAP 버튼(Key 15)의 Rising Edge 검출하여 현재 스탑워치 시간을 **r_lap_time** 배열에 저장
- **r_lap_count**가 1씩 증가하여 최대 8개 까지 저장
- 스탑워치 정지(**r_start_en** = 0)시 **r_lap_count**가 0으로 초기화

5. Lap time led 표시

```
always @(posedge i_clk or negedge i_rstn) begin
    if(!i_rstn) begin
        r_lap_led <= 8'b0000_0000;
    end
    else begin
```

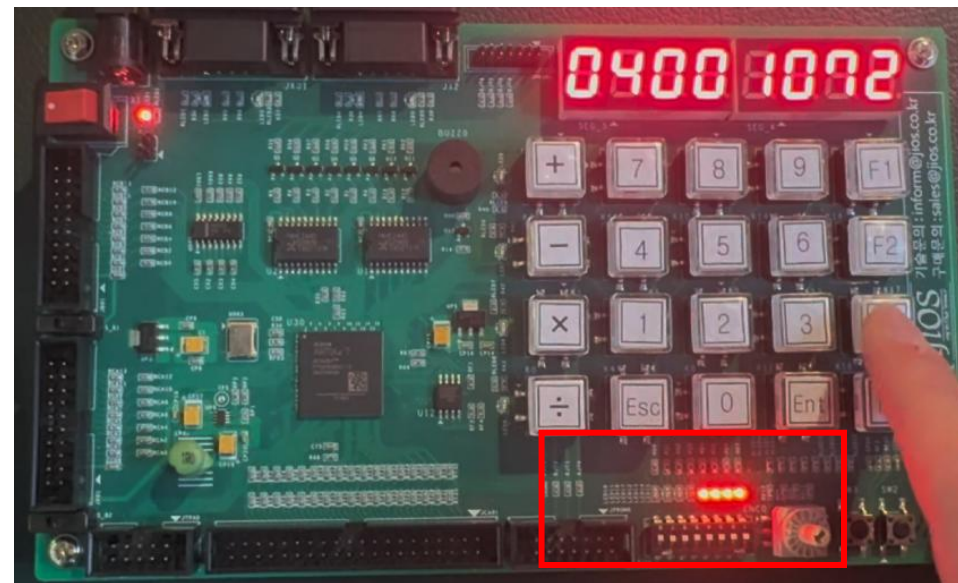
// 저장된 랩 개수만큼 LED 켜기

```
case(r_lap_count)
    3'd0: r_lap_led <= 8'b0000_0000;
    3'd1: r_lap_led <= 8'b0000_0001;
    3'd2: r_lap_led <= 8'b0000_0011;
    3'd3: r_lap_led <= 8'b0000_0111;
    3'd4: r_lap_led <= 8'b0000_1111;
    3'd5: r_lap_led <= 8'b0001_1111;
    3'd6: r_lap_led <= 8'b0011_1111;
    3'd7: r_lap_led <= 8'b0111_1111;
    default: r_lap_led <= 8'b1111_1111;
```

```
endcase
```

```
end
```

```
end
```



- 저장된 lap 개수(r_lap_count)에 따라 led가 순차적으로 점등
- 현재 몇 개의 lap time이 저장되었는지 시각적으로 확인 가능

6. 저장된 Lap time 조회

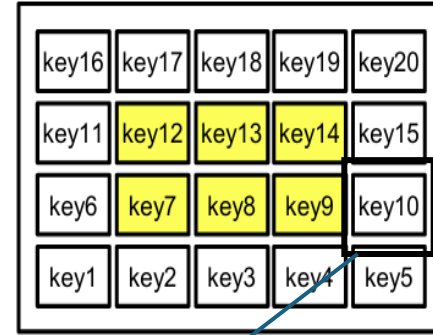
```
(i_lap_view && !r_view_sw_d && i_mode_sw) begin
if (r_lap_count > 0) begin
    if (!r_view_mode) begin
        // 첫 조회: view_mode만 켜고 index는 0 유지
        r_view_mode <= 1;
        r_view_index <= 0;
    end
    else begin
        // 이미 조회 중: 다음 인덱스로 이동
        if (r_view_index >= r_lap_count - 1)
            r_view_index <= 0;
        else
            r_view_index <= r_view_index + 1;
    end
end
end
```

```
assign w_view_lap = r_lap_time[r_view_index];
assign w_lap_ms = w_view_lap % 100;
assign w_lap_sec = (w_view_lap / 100) % 60;
assign w_lap_min = (w_view_lap / 6000);
```

```
assign w_lap_digit_1 = w_lap_ms % 10;
assign w_lap_digit_2 = w_lap_ms / 10;
assign w_lap_digit_3 = w_lap_sec % 10;
assign w_lap_digit_4 = w_lap_sec / 10;
assign w_lap_digit_5 = w_lap_min % 10;
assign w_lap_digit_6 = w_lap_min / 10;
// 조회 모드에서 몇 번째 랩인지 표시 (1~8)
```

```
assign w_lap_digit_7 = r_view_index + 1;
assign w_lap_digit_8 = 4'd0;
```

```
assign w_lap_disp = {w_lap_digit_8, w_lap_digit_7,
                    w_lap_digit_6, w_lap_digit_5,
                    w_lap_digit_4, w_lap_digit_3,
                    w_lap_digit_2, w_lap_digit_1};
```

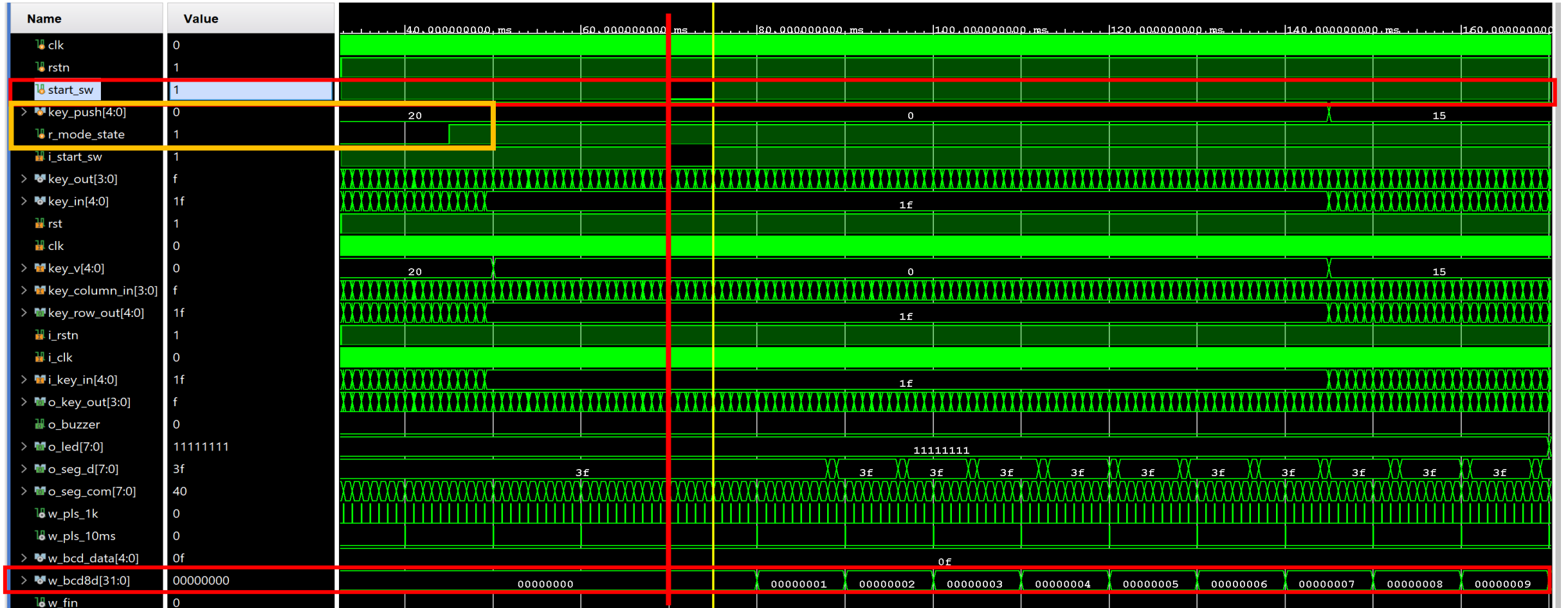


- Key 10 버튼을 누르면 LAP time 조회 모드가 되고 누를 때 마다 인덱스가 증가하여 원하는 LAP time을 조회 가능

- 조회모드에서 7-segment의 표시는 [0] [lap index] [분] [초] [ms]



4. Simulation 검증



- Key_push [4:0]이 20일 때, 즉, key 20번을 누르면 Timer에서 Stopwatch로 모드 변경. 이는, **r_mode_state**가 0→1로 값으로 확인 가능
- Start_sw가 누르면, stopwatch 기능이 활성화 되어 7-segment(**w_bcd8d[31:0]**) 화면에 0부터 count up되는 것을 확인 가능

5. 프로젝트 소감 및 개선점

- 이번 프로젝트를 통해 기존 Timer 기능에 Stopwatch와 Lap 기능을 추가하면서 Verilog를 활용한 하드웨어 설계와 모듈 간 통신 구조에 대한 이해도를 높일 수 있었습니다. 특히 시뮬레이션과 실제 Fpga 보드에서의 검증을 통해 이론과 실제의 차이를 경험할 수 있었으며 실제 설계한대로 시뮬레이션과 Fpga 보드에서 구현되었을 때 큰 성취감을 느낄 수 있었습니다.
- 현재 최대의 8개의 Lap time 저장만 가능한데, 메모리 용량을 확장하여 더 많은 기록을 저장하도록 개선 가능 할 것 같습니다.
- Lap time 삭제 및 초기화 기능 추가로 사용자의 편의성을 향상 시키면 좋을 것 같습니다.
- 분할 시간 기능을 추가하여 구간별로 시간 측정이 가능하도록 기능을 확장하면 좋을 것 같습니다.

6. FPGA 보드 구현

